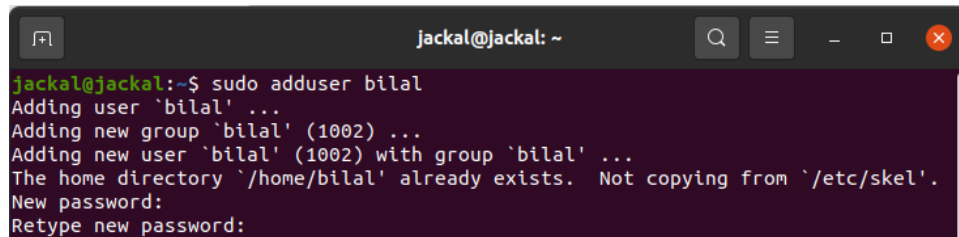


## Lab 2 (b)

### Changing Ownership:

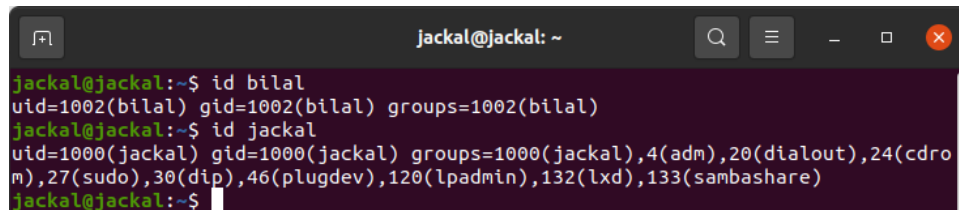
**Creating user:** To understand ownership we need to make another user by executing the following command.

**Command:** `sudo adduser <username>`.



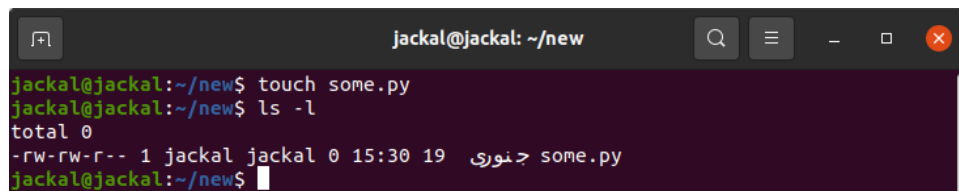
```
jackal@jackal: ~  
jackal@jackal:~$ sudo adduser bilal  
Adding user `bilal' ...  
Adding new group `bilal' (1002) ...  
Adding new user `bilal' (1002) with group `bilal' ...  
The home directory `/home/bilal' already exists. Not copying from `/etc/skel'.  
New password:  
Retype new password:
```

**Checking ID of the user:** Use the `id <username>` command to print the ID assigned to user as below. As we can see bilal has 1002 id and jackal (main user) have 1000 id and other supplementary ids.



```
jackal@jackal: ~  
jackal@jackal:~$ id bilal  
uid=1002(bilal) gid=1002(bilal) groups=1002(bilal)  
jackal@jackal:~$ id jackal  
uid=1000(jackal) gid=1000(jackal) groups=1000(jackal),4(adm),20(dialout),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),132(lxd),133(sambashare)  
jackal@jackal:~$
```

**Changing ownership of a file some.py:** As we have now multiple users, we can change the ownership of the file to another user as below. Let's create a new file and check permissions and ownership.



```
jackal@jackal: ~/new  
jackal@jackal:~/new$ touch some.py  
jackal@jackal:~/new$ ls -l  
total 0  
-rw-rw-r-- 1 jackal jackal 0 15:30 19 ١٥ نوري some.py  
jackal@jackal:~/new$
```

And now change the ownership using following command.

**Command:** `sudo chown <user> <file>`.

```
jackal@jackal: ~/new
jackal@jackal:~/new$ sudo chown bi
bilal bin
jackal@jackal:~/new$ sudo chown bilal some.py
[sudo] password for jackal:
jackal@jackal:~/new$ ls -l
total 0
-rw-rw-r-- 1 bilal jackal 0 15:30 19 ٢٠٢٠ some.py
jackal@jackal:~/new$
```

**Sudo Group:** We can add a user in sudo group as below.

```
jackal@jackal: ~
jackal@jackal:~$ sudo adduser bilal sudo
Adding user 'bilal' to group 'sudo' ...
Adding user bilal to group sudo
Done.
```

**Login to user:** We can login to another user as below.

```
bilal@jackal: ~
jackal@jackal:~$ sudo su - bilal
[sudo] password for jackal:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
bilal@jackal:~$
```

## Changing Permissions:

**Command:** In Linux we use **chmod** command to change the permission of any file for groups. We can change the permission using two ways; **symbolic and octal mode**.

**Symbolic mode:** This method uses operator, letters and group type or reference. Have a look at operators in Table 1, letters in Table 2 and Reference in Table 3.

Table 1.

| Operator | Work                         |
|----------|------------------------------|
| "+"      | Add Permission               |
| "_"      | Remove Permission            |
| "="      | Assign or set the permission |

Table 2.

| Letter | Permission Type |
|--------|-----------------|
| "r"    | Read            |
| "w"    | Write           |
| "x"    | Execute         |

Table 3.

| Reference | Group Type |
|-----------|------------|
| "u"       | Owner      |
| "g"       | Group(s)   |
| "o"       | Others     |
| "a"       | All        |

**Usage:** Let's create a simple shell script with name hello.sh, which only has one line => echo "Hello World", save it and check its permissions as below shown.

```

jackal@jackal: ~/permission
jackal@jackal:~$ mkdir permission
jackal@jackal:~$ cd permission/
jackal@jackal:~/permission$ nano hello.sh
jackal@jackal:~/permission$ ls -l
total 4
-rw-rw-r-- 1 jackal jackal 19 13:35 20 جنوري hello.sh
jackal@jackal:~/permission$

```

**Execution Permission:** We won't be able to run this script as it doesn't have execution permission as shown below.

```

jackal@jackal: ~/permission
jackal@jackal:~/permission$ ls -l
total 4
-rw-rw-r-- 1 jackal jackal 19 13:35 20 جنوري hello.sh
jackal@jackal:~/permission$ ./hello.sh
bash: ./hello.sh: Permission denied
jackal@jackal:~/permission$

```

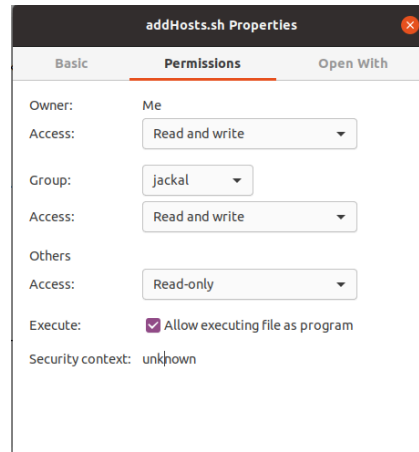
**Change Permission:** Let's give all groups permission for the execution, check the permission and run it as below shown.

```

jackal@jackal: ~/permission
jackal@jackal:~/permission$ sudo chmod a+x hello.sh
jackal@jackal:~/permission$ ls -l
total 4
-rwxrwxr-x 1 jackal jackal 19 13:35 20 جنوري hello.sh
jackal@jackal:~/permission$ ./hello.sh
Hello World
jackal@jackal:~/permission$

```

**Using GUI:** We can also change the permissions of file using GUI of Ubuntu by going to properties of file and then permissions as below.



**Octal Mode:** Uses three-digit octal number to set permissions. The first digit sets the permission for the owner, second digit for group and third digit for others. See the Table 4 for usage of this method.

Table 4.

| Number | Permission(s) |
|--------|---------------|
| 0      | ---           |
| 1      | --X           |
| 2      | -W-           |
| 3      | -WX           |
| 4      | r--           |
| 5      | r-X           |
| 6      | rw-           |
| 7      | rwx           |

**Usage:** Let's change the permissions of hello.sh created earlier as below using octal method. We will give all permissions to the owner, read permission to the group and no permission to others.

```

Jackal@jackal: ~/permission
jackal@jackal:~/permission$ ls -l
total 4
-rwxrwxr-x 1 jackal jackal 19 13:35 20  >نوری hello.sh
jackal@jackal:~/permission$ sudo chmod 740 hello.sh
jackal@jackal:~/permission$ ls -l
total 4
-rwxr----- 1 jackal jackal 19 13:35 20  >نوری hello.sh
jackal@jackal:~/permission$

```

## Compile C code:

**Writing Code:** Below is simple C code. Create file using nano and add name this as hello.c

```
#include <stdio.h>

int main() {
    // This is a simple C program
    printf("Hello, World!\n");
    return 0;
}
```

**Compile:** Compile it using gcc command and produce its executable file.

**Command:** gcc hello.c -o hello

**Execution Command 1:** bash hello.c

**Execution Command 2:** ./hello.c

## Overall Procedure and Output:

```
jackal@jackal:~$ nano hello.c
jackal@jackal:~$ gcc hello.c -o hello
jackal@jackal:~$ ./hello
Hello, World!
jackal@jackal:~$
```